FOODLINK - INTELLIGENT FOOD SAFETY AND DONATION SYSTEM

23IT53C – MODERN WEB TECHNOLOGIES

PROJECT REPORT

Submitted by

INFANT SHERIN VIBI M 2403957620522031

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "**FOODLINK - INTELLIGENT FOOD SAFETY AND DONATION SYSTEM**", is the bonafide work of **INFANT SHERIN VIBI M (2403957620522031)** who carried out the project work under my supervision.

Course Instructor

Submitted to the **23IT53C – MODERN WEB TECHNOLOGIES** Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	3
2	OBJECTIVES	4
3	DESCRIPTION	5
4	CLASS DIAGRAM	6
5	TABLE STRUCTURES	7
6	ER DIAGRAM	10
7	TECH STACK	11
8	MODULES	12
9	CONCLUSION	24

CHAPTER 1

INTRODUCTION

FoodLink is a web-based application developed to reduce food waste by connecting restaurants that have surplus food with NGOs that can collect it and serve it to people in need before it spoils. It is designed to replace the manual, phone-call-based coordination that restaurants and charities usually depend on, and to give every donation a clear, trackable path from the kitchen to the community. The system provides three types of users: Admin, Restaurant, and NGO, each having specific roles and access privileges.

A Restaurant can list surplus food with its category, quantity, number of meals, and expiry time, scan a photo of the food with the AI Food Scanner to judge its freshness, and use the SmartMatch engine to see which NGOs can actually reach the food before it expires. An NGO sees a live feed of food that it can collect in time, sorted by match score, and can claim items, accept offers, and mark donations as picked up and completed. The Admin can monitor the whole platform, view platform-wide statistics and all donations, review the activity log, and enable or disable user accounts, with the change taking effect instantly.

The system helps reduce food waste, saves time for restaurants and charities, and improves transparency by recording every step of every donation. A key feature is SmartMatch, which does not simply list nearby NGOs but checks whether each NGO can physically arrive before the food expires, taking distance, travel speed, handling time, daily capacity, and past reliability into account. The platform also includes a chatbot that answers from live data, a three-language interface (English, Hindi, and Tamil), and an impact estimate showing meals rescued, food saved, and CO_2 emissions avoided. This system is useful for restaurants, bakeries, caterers, and community kitchens that want to donate surplus food in a more organized and efficient manner.

CHAPTER 2

OBJECTIVES

- To provide role-based authentication and authorization using JWT access and refresh tokens, ensuring secure access control for each user type (Admin, Restaurant, NGO).
- To offer Google OAuth 2.0 sign-in as an alternative to email and password registration.
- To maintain a centralized MySQL database for storing all user, organization, food listing, donation, and activity details efficiently.
- To allow restaurants to add, edit, and remove surplus food listings with quantity, meals, category, and expiry time.
- To rank NGOs for each food item using SmartMatch, based on proximity, time safety, capacity headroom, and reliability, and to exclude NGOs that cannot arrive before the food expires.
- To manage each donation through a controlled lifecycle (Pending, Accepted, Picked Up, Completed, Declined, Cancelled) enforced on the server, so that two NGOs can never claim the same food.
- To update food status automatically (Available, Expiring Soon, Expired) and to cancel stale offers on food that has spoiled, without any manual action.
- To provide an AI Food Scanner that identifies food from a photo, judges its freshness, and estimates a safe donation window.
- To provide a chatbot that answers questions about expiring food, donation status, best NGO match, impact, and recently viewed items using live database data.
- To support English, Hindi, and Tamil across the interface, the chatbot, and the scanner, with the chosen language saved to the user's account.
- To give the Admin a dashboard with platform statistics, a 14-day donation chart, leaderboards, an audit log, and the ability to enable or disable users instantly.
- To track recently accessed foods, NGOs, and donations for every user and show them as clickable shortcuts on the dashboards.
- To estimate the social impact of donations in terms of meals rescued, kilograms of food saved, and CO_2 emissions avoided.
- To ensure data validation, error handling, password hashing, and rate limiting on login and registration to protect data accuracy and security.

CHAPTER 3

DESCRIPTION

FoodLink is a web-based application designed to move surplus food from restaurants to NGOs efficiently and safely. The system includes multiple types of users: Admin, Restaurant, and NGO. A Restaurant registers its organization, sets its location, and lists the food it has left over. For every item, FoodLink can show a ranked list of NGOs that are able to collect it before the expiry time, and the restaurant can send an offer to the best choice. An NGO registers with its location, daily meal capacity, service radius, accepted food types, and whether it owns a vehicle. It then sees only the food it can realistically collect in time, and can claim it directly from its feed.

Every donation moves through a defined lifecycle: Pending, Accepted, Picked Up, and Completed, with Declined and Cancelled as the alternative endings. The server decides who may make each move and performs the change inside a database transaction with row locking, so the data stays consistent even when several users act at the same time. A background expiry job runs every minute to keep food statuses truthful and to cancel offers on food that has expired. The Admin has a separate login, a dashboard of platform-wide statistics, and tools to manage users. All information is stored securely in a MySQL database, and access is controlled through a login page with JWT-based sessions. This project helps in reducing food waste, reducing manual coordination, and maintaining organized digital records of donations.

CHAPTER 4

CLASS DIAGRAM

The Users class is the root of the model. Every Restaurant and every NGO has exactly one linked user account, and each user can own many recently accessed records, refresh tokens, and activity log entries. A Restaurant lists many Food_Items, and each Food_Item can be offered in many Donations. An NGO receives many Donations. The Donations class therefore joins a food item with the NGO that takes it, and it carries the status, the SmartMatch score, and the number of servings delivered.



Fig 4.1 – Class Diagram – Classes (tables), their attributes, and the one-to-one and one-to-many relationships between them.

CHAPTER 5

TABLE STRUCTURES

Users

The Users table stores the basic details of all users registered in the FoodLink system, including their name, email, password, and role.It supports different roles such as Admin, Restaurant, and NGO for role-based access.The table also stores Google authentication details, preferred language, account status, and last login information.Unique constraints and timestamps help maintain secure, active, and properly tracked user accounts.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(120)	NO		NULL	
email	varchar(190)	NO	UNI	NULL	
password_hash	varchar(255)	YES		NULL	
role	enum('admin', 'restaurant', 'ngo')	NO		NULL	
google_id	varchar(64)	YES	UNI	NULL	
avatar_url	varchar(500)	YES		NULL	
language	enum('en','hi', 'ta')	NO		en	
is_active	tinyint(1)	NO		1	
last_login_at	datetime	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Restaurants

The Restaurants table stores the details of restaurants registered in the FoodLink system, including name, address, city, and contact information.It also maintains the restaurant's latitude and longitude to support location-based food donation services.The user_id uniquely links each restaurant with its corresponding user account.The table also records the creation date of each restaurant profile.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	YES	UNI	NULL	
name	varchar(150)	NO		NULL	
address	varchar(255)	YES		NULL	
city	varchar(100)	YES		NULL	
latitude	decimal(9,6)	YES		NULL	
longitude	decimal(9,6)	YES		NULL	
phone	varchar(25)	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Food Items

The Food Items table stores detailed information about food items listed by restaurants for donation, including the food name, category, quantity, unit, and number of servings. The restaurant_id links each food item to the respective restaurant. It also tracks the expiry time and status, such as Available, Expiring Soon, Reserved, Donated, or Expired. Additional notes can be added to provide information about the food. The created_at and updated_at fields record when the food item was created and last modified.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
restaurant_id	int	NO	MUL	NULL	
name	varchar(150)	NO		NULL	
category	enum('cooked', 'bakery', 'produce','dairy', 'packaged')	NO		cooked	
quantity	decimal(8,2)	NO		NULL	
unit	enum('kg','litres', 'packs','plates')	NO		kg	
servings	int	NO		1	
expiry_time	datetime	NO		NULL	
status	enum('Available', 'Expiring Soon', 'Reserved', 'Donated', 'Expired')	NO	MUL	Available	
notes	varchar(255)	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
Updated at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Donations

The Donations table manages the food donation transactions between restaurants and NGOs in the FoodLink system. It links each donation to a specific food item and NGO using food_id and ngo_id. The initiated_by field identifies whether the donation was initiated by a restaurant or NGO, while the status tracks stages such as Pending, Accepted, Picked Up, Completed, Declined, or Cancelled. The table also stores the match score and servings delivered to monitor donation suitability and quantity. The created_at and updated_at fields maintain the donation transaction history and update records automatically.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
food_id	int	NO	MUL	NULL	
ngo_id	int	NO	MUL	NULL	
initiated_by	enum('restaurant', 'ngo')	NO		NULL	
status	enum('Pending', 'Accepted','Picked Up','Completed', 'Declined', 'Cancelled')	NO		Pending	
match_score	decimal(5,2)	YES		NULL	
servings_delivered	int	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP

Recently Accessed

The Recently Accessed table records the entities that users have recently viewed or interacted with in the FoodLink system. It links each record to a specific user through user_id and identifies the accessed entity as food, NGO, or donation using entity_type. The entity_id and title fields store the corresponding entity details for easy retrieval. The accessed_at field records the exact date and time when the entity was accessed, supporting a recently accessed feature for users.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
entity_type	enum('food','ngo', 'donation')	NO		NULL	
entity_id	int	NO		NULL	
title	varchar(190)	NO		NULL	
accessed_at	datetime	NO		NULL	

Refresh Tokens

The Refresh Tokens table manages secure authentication sessions for users in the FoodLink system. The user_id field links each refresh token to the corresponding user account. The token_hash field stores a unique hashed version of the refresh token to improve security. The expires_at field records the date and time when the token becomes invalid. The created_at field records when the token was generated. Overall, this table supports secure session management and token-based authentication.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
token_hash	char(64)	NO	UNI	NULL	
expires_at	datetime	NO		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Activity Log

The Activity Logs table maintains a record of important actions performed by users in the FoodLink system. The user_id field identifies the user associated with each activity. The action field stores the type of operation performed, such as login, donation, or food management. The details field provides additional information about the recorded activity. The created_at field records the exact date and time when the action occurred. This table helps administrators monitor and track user activities throughout the system. It also supports auditing, security monitoring, and maintaining a reliable activity history.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	YES		NULL	
action	varchar(60)	NO		NULL	
details	varchar(255)	YES		NULL	
created_at	timestamp	NO	MUL	CURRENT_TIMESTAMP	DEFAULT_GENERATED

CHAPTER 6

ER DIAGRAM

This ER diagram represents the FoodLink food donation management system, showing the entities, attributes, and relationships involved in the platform.The Users entity maintains user information and is connected to Restaurant and NGO profiles through the has profile relationship.A Restaurant can list multiple Food Items, while each food item can be associated with donation activities through the Donation entity.The Donation entity connects food items with NGOs and stores details such as initiation type, status, and match score. The diagram also includes Recently Accessed, Refresh Token, and Activity Log entities to manage user access history, authentication sessions, and system activities.Cardinality indicators such as 1 and N define one-to-one and one-to-many relationships between the entities.Overall, the diagram provides a clear structural representation of how users, restaurants, NGOs, food items, and donations interact within FoodLink.

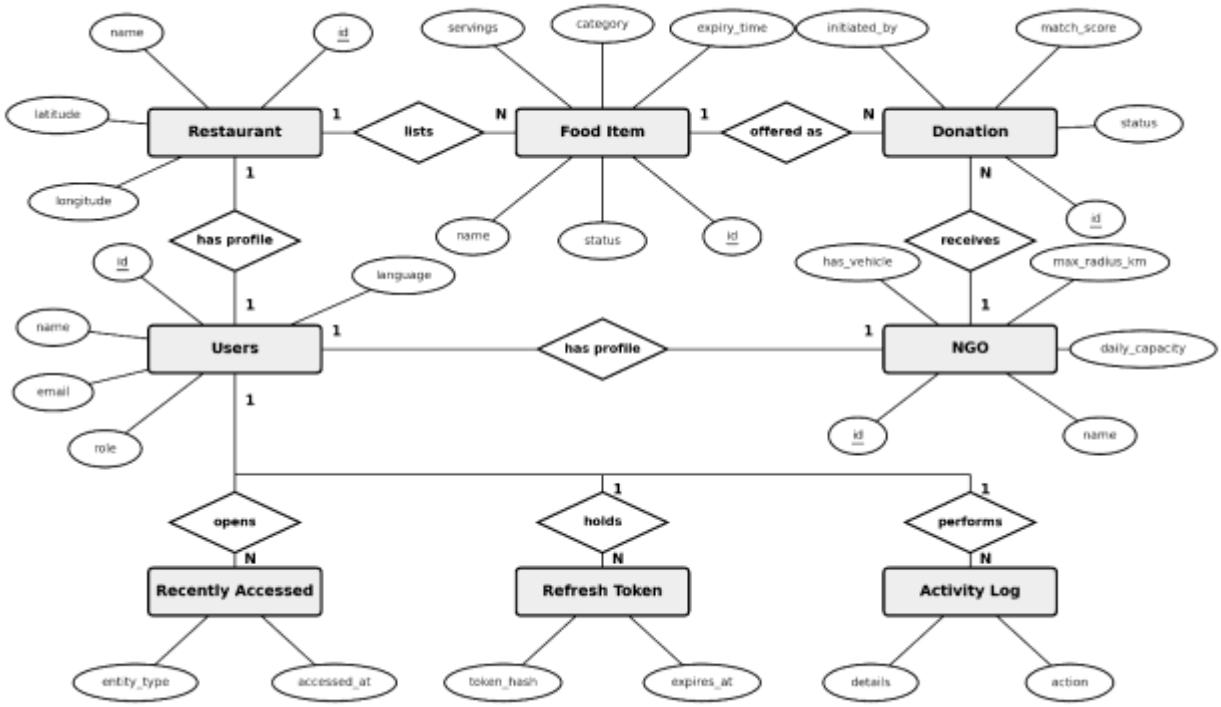


Fig 6.1 – ER Diagram – Entities, attributes, and relationships of the FoodLink database.

In the ER diagram, a User has one profile, either a Restaurant or an NGO, depending on its role. A Restaurant lists many Food Items (1 : N). A Food Item can be offered as many Donations over time (1 : N), for example when one NGO declines and another accepts. An NGO receives many Donations (1 : N). Each User also opens many Recently Accessed items, holds many Refresh Tokens, and performs many actions recorded in the Activity Log.

CHAPTER 7

TECH STACK

Technologies and Architecture

The FoodLink application is developed as a full-stack web-based food donation management system using modern frontend, backend, database, security, and AI technologies. The system follows a modular MVC (Model–View–Controller) architecture with an additional service layer, which helps separate user interface components, business logic, database operations, and external AI services. This structure makes the application easier to maintain, test, extend, and debug.

Frontend Technologies

The frontend of FoodLink is developed using React.js 18 and Vite 5. React provides a component-based approach for building dynamic and reusable user interface elements. Different pages and interface components are organized into reusable React components, allowing the system to provide separate interfaces for different types of users such as administrators, restaurants, and NGOs. React Router 6 is used to manage client-side navigation between different pages without requiring complete page reloads. CSS is used to design and style the application interface, providing a consistent and responsive user experience.

Vite 5 is used as the frontend development and build tool. It provides fast development-server startup and efficient hot-module replacement during development. The Vite development proxy is also configured to forward API requests from the frontend to the backend server, making frontend-backend integration easier during development.

Backend Technologies

The backend is implemented using Node.js with ES modules and Express.js 4. Node.js provides the runtime environment for executing JavaScript on the server, while Express.js handles HTTP requests, routing, middleware processing, and REST API development. The backend follows a modular structure where routes, controllers, middleware, and services are separated according to their responsibilities.

The application exposes RESTful APIs through Express.js for operations such as user authentication, restaurant management, NGO management, food item management, donations, activity tracking, and other system functions. Asynchronous operations are implemented using Async/Await programming, which makes database operations and external API calls easier to manage while maintaining readable code.

Middleware and Request Processing

Several middleware components are used to improve the functionality and security of the backend. JWT authentication middleware verifies access tokens and ensures that protected

resources can only be accessed by authenticated users. Role-Based Authorization middleware controls access according to the user's role, such as administrator, restaurant, or NGO.

The system also uses Helmet middleware to add appropriate HTTP security headers and reduce common web security risks. CORS middleware controls cross-origin requests between the React frontend and Express backend. A rate-limiting middleware is used to restrict excessive requests and provide additional protection against abuse. The JSON body parser processes JSON request data received from the frontend and makes it available to backend route handlers.

Database Technology

MySQL is used as the primary relational database for storing and managing FoodLink data. The application uses the mysql2 driver to establish communication between the Node.js backend and MySQL database. A connection pool is used to efficiently manage multiple database connections and improve database performance.

The database contains tables for important entities such as users, restaurants, NGOs, food items, donations, recently accessed records, refresh tokens, and activity logs. Relationships between these entities allow the system to connect users with their restaurant or NGO profiles, associate food items with restaurants, and manage donations between restaurants and NGOs.

The application uses parameterized raw SQL queries for database operations. This approach helps prevent SQL injection attacks while providing direct control over database queries. Database transactions are used when multiple related operations must be completed together. Row-level locking is also used where necessary to prevent conflicting operations when multiple users or processes attempt to modify the same database records simultaneously.

Security and Authentication

Security is an important part of the FoodLink system. JSON Web Tokens (JWT) are used to implement authentication using access and refresh tokens. Access tokens are used to authenticate API requests, while refresh tokens support maintaining secure user sessions without requiring users to log in repeatedly.

User passwords are protected using bcryptjs password hashing, ensuring that plain-text passwords are not stored directly in the database. The system also supports Google OAuth 2.0 authorization-code flow, allowing users to authenticate through their Google accounts. This provides an additional sign-in mechanism alongside traditional email and password authentication.

Role-based access control ensures that users can access only the functionality appropriate to their assigned roles. For example, restaurant users can manage their food listings

and donations, while NGO users can access relevant food donation information and manage their donation activities.

AI Services

FoodLink integrates Anthropic Claude API to provide artificial intelligence capabilities within the application. One major AI feature is the AI Food Scanner, which performs image analysis to identify and analyze food items. This feature can assist users in understanding the type and condition of food and supports the food donation workflow.

The Claude API is also used for the application's open-ended chatbot. The chatbot allows users to interact with the system using natural-language questions and receive AI-generated responses. These AI services are implemented through a dedicated service layer so that external API communication remains separate from the main controller and database logic.

Development Approach

The application follows several software development practices to maintain code quality and security. Parameterized SQL queries are used for safe database communication, while Async/Await programming is used for asynchronous backend operations. The system follows RESTful API design principles, allowing the frontend and backend to communicate through well-defined HTTP endpoints.

The backend is organized using a modular architecture, where authentication, food management, donations, users, NGOs, restaurants, chatbot functionality, and other features are separated into appropriate modules. This reduces code duplication and makes individual features easier to modify or extend.

Development Tools

The main development environment is Visual Studio Code (VS Code). Nodemon is used during backend development to automatically restart the Node.js server whenever source files are modified. Concurrently allows multiple development processes, such as the frontend and backend servers, to be started together. Vite provides the frontend development server and development proxy for communication with the backend.

These tools improve development efficiency by reducing manual restart operations and providing a convenient environment for developing and testing the complete FoodLink application.

MVC Architecture with Service Layer

FoodLink follows an MVC architecture enhanced with a service layer. The Model layer represents the application's database structure. MySQL tables such as users, restaurants, ngos,

food_items, and donations store the application's persistent data. These tables are accessed through parameterized SQL queries implemented within controllers and services.

The View layer is implemented using React components and pages. It is responsible for displaying information, accepting user input, handling user interactions, and communicating with backend APIs. React's component-based structure allows commonly used interface elements to be reused across different pages.

The Controller layer consists of Express route handlers that receive API requests, validate and process input, invoke the required operations, and return appropriate API responses. Controllers act as an intermediary between the frontend requests and the application's business logic.

The Service layer contains specialized business logic that should remain separate from the controllers. Important services include the SmartMatch engine, which supports matching food donations with suitable NGOs; the expiry job, which manages food expiry-related processing; the chatbot service, which handles AI-based conversational functionality; and the Anthropic API client, which manages communication with Claude services.

Overall, this technology stack and architecture provide FoodLink with a scalable, secure, modular, and maintainable full-stack structure. The combination of React, Node.js, Express, MySQL, JWT authentication, Google OAuth, and AI services enables the system to manage food donations efficiently while providing secure user access and intelligent features.

CHAPTER 8

MODULES

8.1 FOLDER STRUCTURE

BACKEND FOLDER STRUCTURE

The FoodLink backend structure follows a modular and organized architecture that separates different responsibilities of the application. The server.js file acts as the main entry point, while the config folder manages database and environment configurations. The controllers handle business logic for authentication, food items, donations, NGOs, dashboards, chatbot, scanner, and administration. The db folder contains database schema, setup, and seed files, while middleware provides authentication and authorization functionality.

```
foodlink/
└── backend/
    ├── server.js
    ├── package.json
    ├── .env.example
    ├── config/
    │   ├── db.js
    │   └── env.js
    ├── controllers/
    │   ├── authController.js
    │   ├── foodController.js
    │   ├── donationController.js
    │   ├── ngoController.js
    │   ├── dashboardController.js
    │   ├── chatController.js
    │   ├── scannerController.js
    │   └── adminController.js
    ├── db/
    │   ├── schema.sql
    │   ├── setup.js
    │   └── seed.js
    ├── middleware/
    │   └── auth.js
    ├── routes/
    │   ├── authRoutes.js
    │   ├── foodroutes.js
    │   ├── donationRoutes.js
    │   └── featureRoutes.js
    ├── services/
    │   ├── matchEngine.js
    │   ├── expiryJob.js
    │   ├── chatbot.js
    │   └── anthropic.js
    ├── tests/
    │   ├── loginCheck.js
    │   └── workflowCheck.js
    └── utils/
        ├── tokens.js
        ├── validate.js
        ├── recent.js
        └── activity.js
```

Fig 8.1 – Backend Folder Structure – Overall organization of backend files and directories showing modular architecture.

BACKEND ROUTES

In a Node.js and Express-based application, the routes, services, and controllers together form the backbone of the backend architecture, ensuring modularity, readability, and maintainability of the code. **Routes** define the various endpoints (URLs) through which clients can interact with the server. Each route corresponds to a specific action, such as creating, reading, updating, or deleting data. The route files handle HTTP methods like GET, POST, PUT, PATCH, and DELETE, apply the authentication and role middleware, and connect the incoming client requests to the appropriate controller functions.

Controllers act as the link between the routes and the business logic. They validate the incoming input, interact with the database, and send the appropriate response back to the client. Controllers keep the route files clean and focus on operations such as checking ownership, managing transactions, and formatting responses.

Services form the business logic layer that handles the complex operations: the SmartMatch scoring engine, the background expiry job, the chatbot, and the Anthropic API client. By placing these operations in the service layer, the controllers remain lightweight and the same logic is reused across several controllers. For example, the SmartMatch scoring function is used by the restaurant's ranking view, by the NGO's feed, by the claim and offer actions, and by the chatbot.

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a Restaurant or NGO account
POST	/api/auth/login	Public	Sign in; returns access and refresh tokens
POST	/api/auth/refresh	Refresh token	Rotate the refresh token and issue a new pair
POST	/api/auth/logout	Refresh token	Revoke the refresh token
GET	/api/auth/google	Public	Start Google OAuth sign-in
GET/PATCH	/api/auth/me	Logged in	View or update own profile and language
GET/POST	/api/food	Logged in / Restaurant	List food (role-based view) or add a listing
GET/PUT/DELETE	/api/food/:id	Logged in / Restaurant	View, edit, or remove a listing
GET	/api/food/:id/matches	Restaurant, Admin	SmartMatch ranking of NGOs for a food item
POST	/api/food/:id/offer	Restaurant	Offer a food item to a chosen NGO
GET	/api/donations	Logged in	List donations visible to the user
POST	/api/donations/claim	NGO	Claim food directly from the ranked feed
PATCH	/api/donations/:id/status	Restaurant, NGO	Move a donation to its next status
GET	/api/ngos, /api/ngos/:id	Restaurant, Admin	List or view NGO details
GET	/api/dashboard/summary, /recent	Logged in	Dashboard counts, impact, recently accessed
POST	/api/chat	Logged in	Chatbot reply in the chosen language
POST	/api/scanner/analyze	Restaurant	AI analysis of an uploaded food photo
GET/PATCH	/api/admin/...	Admin	Statistics, users, activity, all donations, enable/disable user

Table 8.2 – Backend Routes – RESTful API routes connecting client requests to backend controllers.

FRONTEND FOLDER STRUCTURE

The FoodLink application is built with a clear separation of concerns, featuring a Backend API Server and a Frontend User Interface. This architecture is standard for modern web applications and promotes modularity, independent development, and scalability.

```
foodlink/
├── index.html
├── vite.config.js
├── package.json
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── api/
    │   └── client.js
    ├── context/
    │   └── AuthContext.jsx
    ├── i18n/
    │   ├── I18nContext.jsx
    │   └── translations.js
    ├── components/
    │   ├── Navbar.jsx
    │   ├── ProtectedRoute.jsx
    │   ├── FoodForm.jsx
    │   ├── FoodTable.jsx
    │   ├── MatchModal.jsx
    │   ├── DetailModal.jsx
    │   ├── DonationList.jsx
    │   ├── RecentlyAccessed.jsx
    │   ├── ChatBot.jsx
    │   ├── Modal.jsx
    │   ├── StatCard.jsx
    │   ├── StatusBadge.jsx
    │   └── FoodCard.jsx
    └── pages/
        ├── Home.jsx
        ├── Login.jsx
        ├── Register.jsx
        ├── OAuthCallback.jsx
        ├── Dashboard.jsx
        ├── RestaurantDashboard.jsx
        ├── NgoDashboard.jsx
        ├── AdminDashboard.jsx
        ├── FoodScanner.jsx
        └── Profile.jsx
```

Fig 8.3 – Frontend Folder Structure – Layout of the React-based frontend directories with components and pages.

1. Backend API Server Module

The backend, located in the foodlink/backend directory, handles all server-side logic, data management, and security. It is a Node.js/Express application connected to MySQL.

- **API Routing (routes):** Defines all the REST API endpoints that the frontend uses to communicate with the server. Routes are grouped by feature (auth, food, donations, NGOs, dashboard, chat, scanner, admin) and protected by role-based middleware.
- **Business Logic (controllers):** Executes the core business logic for each API request, including input validation, ownership checks, and database transactions.

- **Services (services):** Contains the SmartMatch engine (matchEngine.js), the expiry job (expiryJob.js), the chatbot (chatbot.js), and the Anthropic API client (anthropic.js).
- **Database Module (config/db.js, db/schema.sql, db/setup.js, db/seed.js):** Creates the connection pool, defines the tables, and populates the database with realistic starting records for setup and testing.
- **Middleware (middleware/auth.js):** Verifies the JWT on every protected request and re-checks the account in MySQL, so an Admin disabling a user takes effect immediately; also provides the requireRole check.
- **Utilities (utils):** Reusable helpers for token issue and rotation, input validation, recently accessed tracking, and activity logging.
- **Server Core (server.js, config/env.js):** The application entry point; it loads the environment variables, applies Helmet and CORS, mounts the routers, checks the database connection, and starts the expiry job.

2. Frontend User Interface Module

The frontend, found in the foodlink/src directory, is a Single-Page Application (SPA) built with React and Vite, responsible for presentation and interaction.

- **Pages (pages):** The top-level screens of the application, such as Home, Login, Register, the Restaurant, NGO, and Admin dashboards, the Food Scanner, and Profile.
- **UI Components (components):** Small reusable elements such as the navigation bar, food table, food form, SmartMatch modal, donation list, status badges, stat cards, and the chatbot.
- **API Client (api/client.js):** Centralizes all calls to the backend. It attaches the JWT to each request and, on an expired token, silently exchanges the refresh token for a new pair and retries the request once.
- **State and Language (context, i18n):** AuthContext keeps the signed-in user; I18nContext and translations.js provide the English, Hindi, and Tamil text.
- **Routing (App.jsx, ProtectedRoute.jsx):** Maps URLs to pages and decides which routes need a signed-in user of a particular role.

8.2 USER INTERFACE MODULES

HOME PAGE



Fig 8.4 – Home Page – Landing page introducing FoodLink and its main features.

The Home page introduces FoodLink with the headline "Reduce Food Waste. Feed Communities." and a short description of the platform. It shows the six-step flow of a donation: the restaurant lists surplus food, the scanner checks freshness, expiry is monitored automatically, SmartMatch ranks NGOs that can arrive in time, the NGO accepts and collects, and the donation is completed and counted. A "Why FoodLink?" section highlights the four main features: SmartMatch, the AI Food Scanner, live expiry tracking, and the three-language interface.

REGISTRATION AND LOGIN

The Register page lets a new user choose whether they are a Restaurant or an NGO, and enter their name, organization name, email, and password. The password must contain at least eight characters with a letter and a number. The Login page accepts an email and password, and a "Continue with Google" button starts the Google OAuth 2.0 flow; a Google email that matches an existing account is linked to it. The Admin signs in through a separate page at /admin/login, which accepts only administrator accounts. After a successful login the server issues a 15-minute access token and a 7-day refresh token, and the user is taken to the dashboard for their role. Login and registration requests are rate-limited.

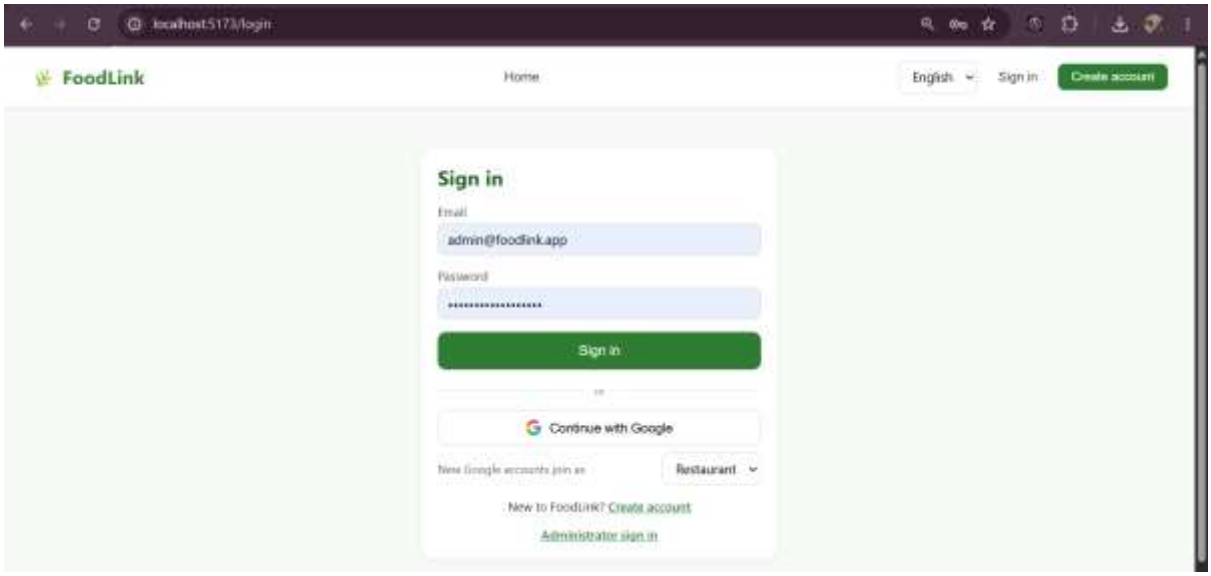


Fig 8.5 – Registration and Login Pages – Account creation and sign-in for Restaurants and NGOs.

RESTAURANT DASHBOARD

The Restaurant Dashboard is the main working screen for a restaurant. At the top it shows the restaurant name with the "Add food" and "Scan food" buttons. A row of stat cards displays Total items, Available, Expiring soon, Reserved, Donated, and Expired counts. Below it, an impact strip shows Meals rescued, Food saved (kg), and CO_2e avoided (kg), labelled clearly as estimates based on 0.4 kg per meal and 2.5 kg CO_2e per kg of food. The Food inventory table lists each item with its category, quantity, meals, a live "Expires in" countdown, and its status, together with the actions Find NGO, Edit, and Remove. The page refreshes every minute so countdowns and statuses stay current. Below the table, the Donations panel and the Recently accessed panel are shown. If the restaurant has not yet set its location, a warning banner links to the Profile page, since SmartMatch needs the location.

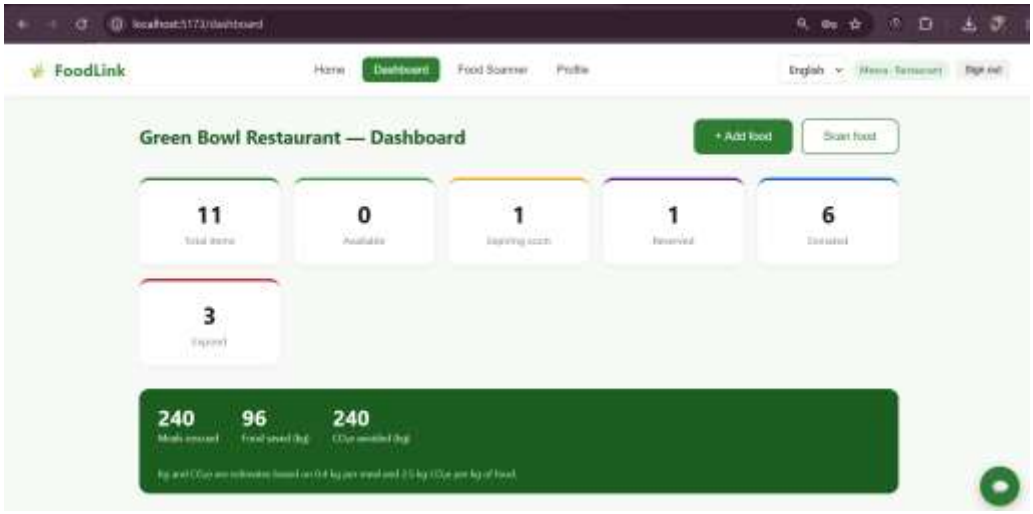


Fig 8.6 – Restaurant Dashboard – Inventory overview with status cards and impact summary.

ADD / EDIT FOOD

The “Add surplus food” form collects the food name, category (Cooked meals, Bakery, Fresh produce, Dairy, or Packaged), quantity, unit (kg, litres, packs, or plates), the number of meals, the expiry date and time, and optional notes. The server validates every field: the name must be present, the category and unit must be valid, the quantity must be greater than zero, the meals must be a whole number of at least one, and the expiry time must be in the future. The status of a new item is calculated automatically from its expiry time. The same form is used to edit an item, and it can be pre-filled by the AI Food Scanner.

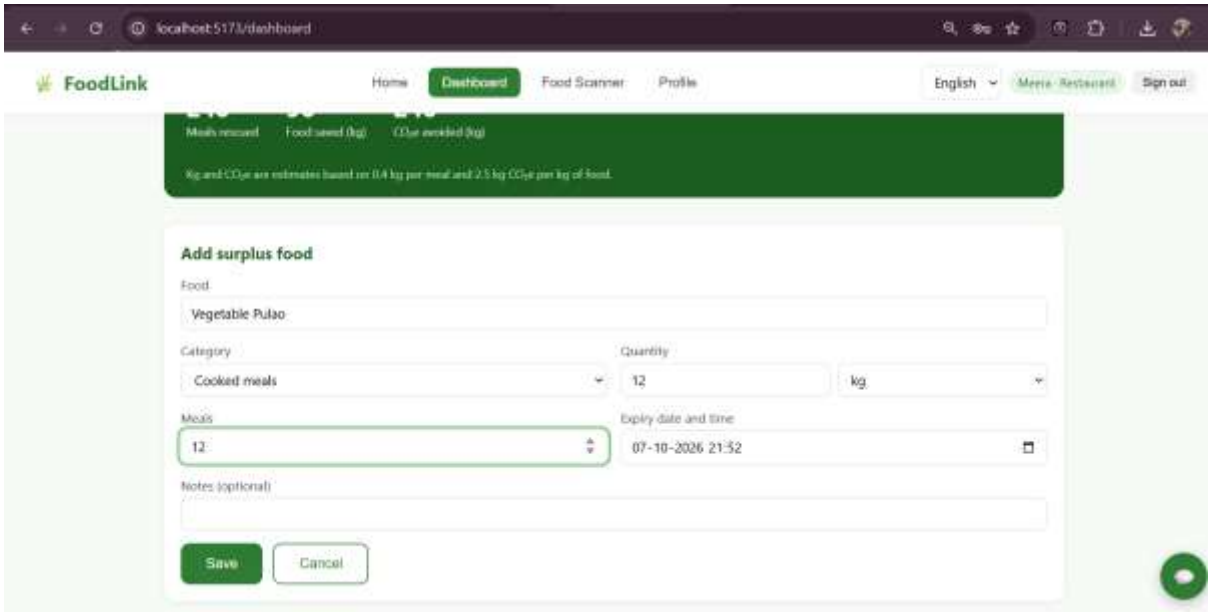


Fig 8.7 – Add Surplus Food Form – Form used to list or edit a surplus food item.

AI FOOD SCANNER

The AI Food Scanner is available to restaurant accounts. The restaurant uploads a photo (JPEG, PNG, or WebP, up to 5 MB), which the browser resizes before sending it to the server. The server passes the image to the Anthropic Claude API with a strict instruction to return a single JSON result. The result shows the detected foods with a confidence percentage, the category, a freshness verdict (Fresh, Use soon, or Spoiled), an estimated safe window in hours, whether the food is safe to donate, a storage tip, and a short explanation of what the AI saw. The scanner is deliberately conservative: if freshness cannot be judged from the image, it answers “Use soon” with a short window. The text is returned in the language chosen by the user. A “Add to inventory” button opens the food form pre-filled with the detected name, category, and expiry time. The page reminds the user that AI estimates can be wrong and the food should always be checked by a person. If no API key is configured, the scanner shows a clear message and the rest of the application is unaffected.

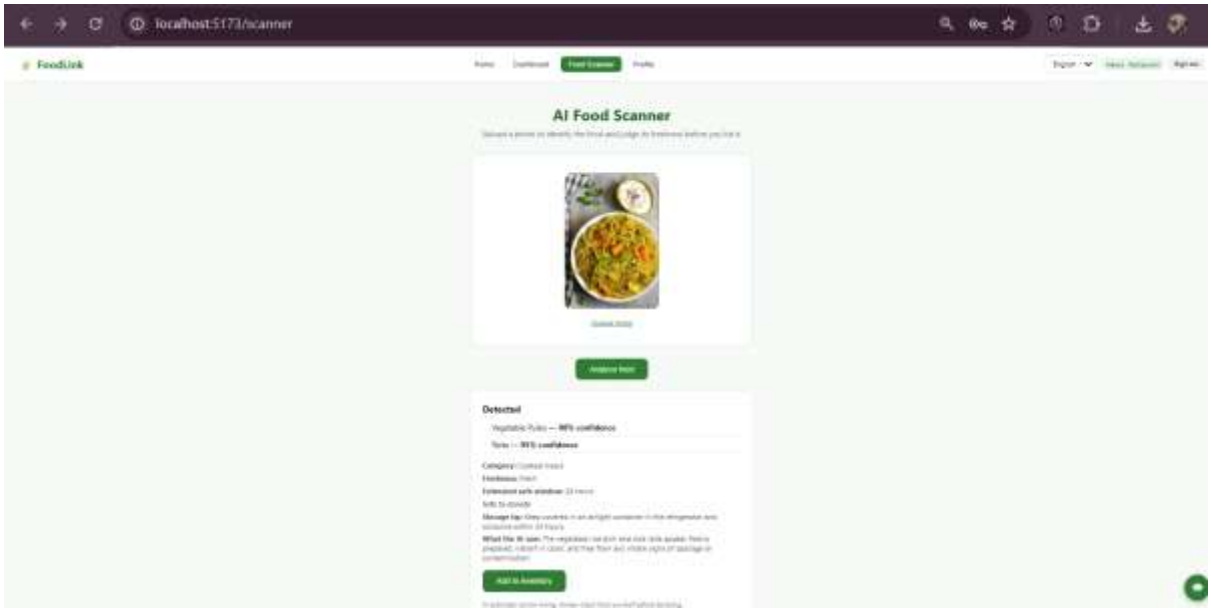


Fig 8.8 – AI Food Scanner Page – Photo-based food identification and freshness check.

SMARTMATCH – UNIQUE FEATURE

SmartMatch answers one question for every food item: which NGO can get this food to people before it spoils? When the restaurant clicks “Find NGO”, the server evaluates every active NGO in two stages. In the first stage, hard filters remove any NGO that cannot take the item, and the reason is shown to the user. In the second stage, the remaining NGOs receive a score from 0 to 100.The travel time is calculated from the straight-line (haversine) distance between the restaurant and the NGO, divided by an average urban speed of 30 km/h for an NGO with a vehicle or 12 km/h on foot, plus 20 minutes for packing and loading. An NGO is feasible only if this travel time is shorter than the time left before expiry

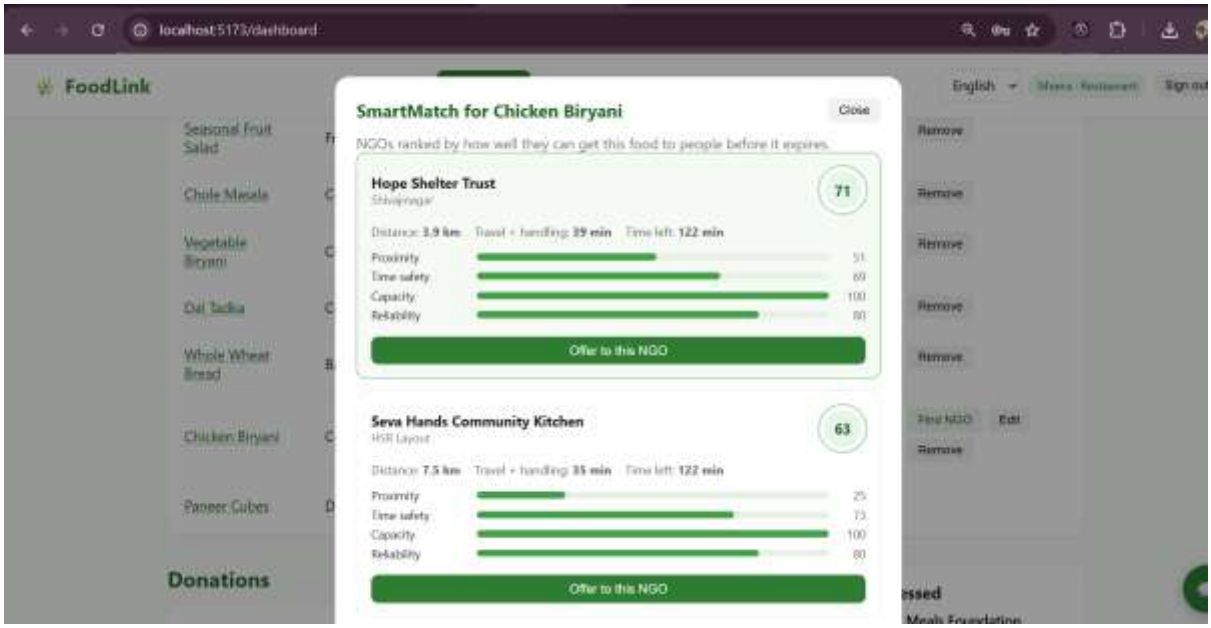


Fig 8.9 – SmartMatch Modal – NGOs ranked for one food item, with the reason behind each score.

.

Excluded because	Meaning
Location missing	The restaurant or the NGO has not set its coordinates
Category not accepted	The NGO does not accept this type of food
Outside service radius	The distance is greater than the NGO's maximum radius
Cannot arrive in time	Travel plus handling time is longer than the time left before expiry
Capacity full	The NGO has no meals of daily capacity left

Score factor	Weight	How it is calculated
Proximity	35%	1 − (distance ÷ the NGO's service radius)
Time safety	25%	Spare minutes before expiry ÷ 120 minutes (2 hours of margin gives full marks)
Capacity headroom	20%	The NGO's remaining meals today ÷ the meals in this item
Reliability	20%	(Completed + 1) ÷ (Completed + Declined + 2), so a new NGO starts at 50%

Each NGO card in the modal shows the match score, the distance, the travel time, the time left, and the four factor scores, together with an "Offer to this NGO" button. NGOs that were excluded are listed separately with their reason. The weights are constants at the top of matchEngine.js, so they can be tuned easily.

NGO DASHBOARD

The NGO Dashboard shows five stat cards: Open food nearby, Awaiting your reply, In progress, Completed, and Meals rescued. A capacity bar shows the meals committed today out of the NGO's daily capacity. The main section, "Food you can collect in time", is the ranked feed: food the NGO cannot reach before it expires is never shown, and the rest is sorted by SmartMatch score. Each card shows the food name, the restaurant, the match score, the category, the number of meals, the distance, the travel time, and the time left, with a "Claim" button. When an NGO claims an item, the server locks the food row, re-checks that it is still available and still feasible for that NGO, and only then creates the donation, so two NGOs can never claim the same food. Below the feed, the My donations panel lets the NGO accept or decline offers and mark donations as picked up or completed, next to the Recently accessed panel.

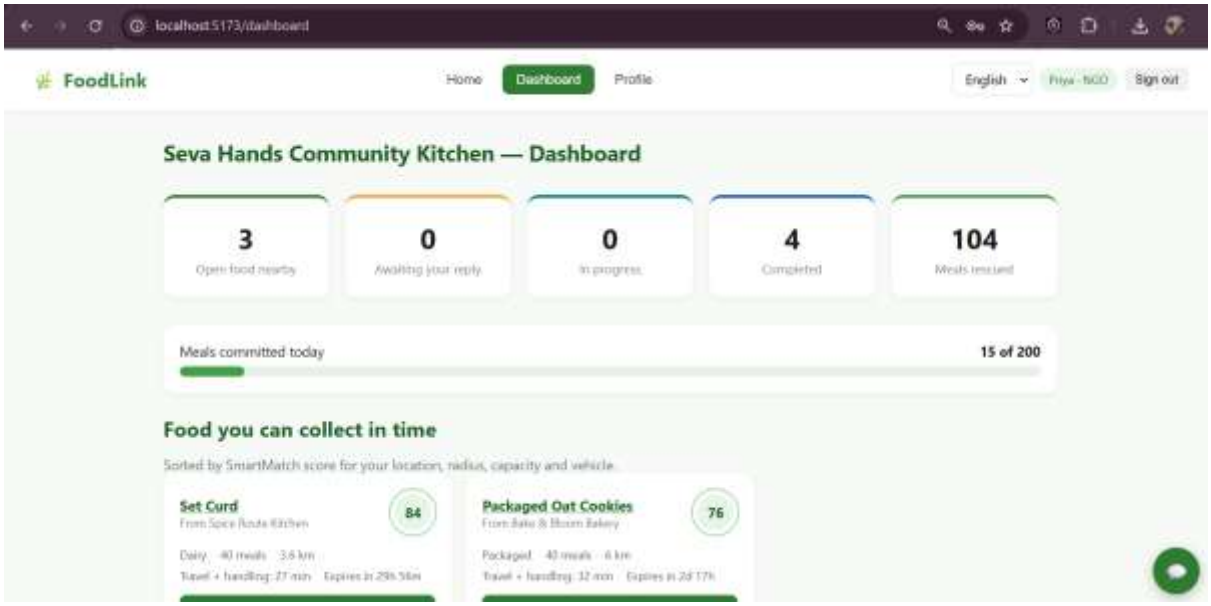


Fig 8.10 – NGO Dashboard – Ranked feed of food the NGO can collect in time.

DONATION LIFECYCLE

Every donation follows a state machine that is enforced on the server inside a database transaction with a row lock. A restaurant's offer creates a Pending donation, and an NGO's claim from its feed creates an Accepted donation directly. The NGO can accept or decline a Pending offer, and mark an Accepted donation as Picked Up. Either side can complete a Picked

Up donation. The restaurant can cancel a Pending or Accepted donation, and the NGO can cancel an Accepted one. Any other change is rejected with a clear message. When a donation is Completed, the food becomes Donated and the number of servings delivered is recorded; when it is Declined or Cancelled, the food returns to Available or Expiring Soon according to its expiry time. A background job runs every minute: it cancels open offers on food that has expired, marks spoiled food as Expired, and changes food that expires within 24 hours to Expiring Soon.

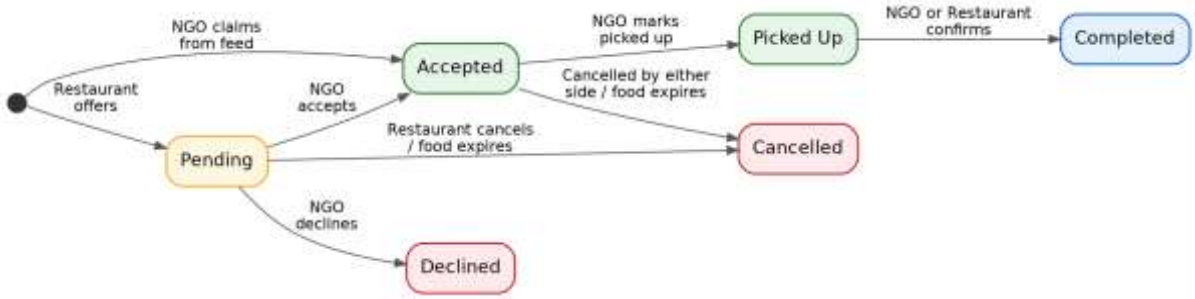


Fig 8.11 – Donation Lifecycle – Allowed status changes and who may make them.

ADMIN DASHBOARD

The Admin signs in at /admin/login and sees the Administrator dashboard. It shows platform-wide cards for Users, Restaurants, NGOs, Food items, Donations, Meals rescued, Completed, and In progress, and a chart of donations over the last 14 days. Two leaderboards list the top donating restaurants and the top receiving NGOs by meals. The Manage users table lists each user with their email, role, organization, last sign-in, and access status, and a button to Disable or Enable the account; a disabled user is rejected on the very next request, even with a valid token. The All donations panel shows every donation with its status, and the Recent activity panel is an audit log of actions such as offers, claims, and status changes.

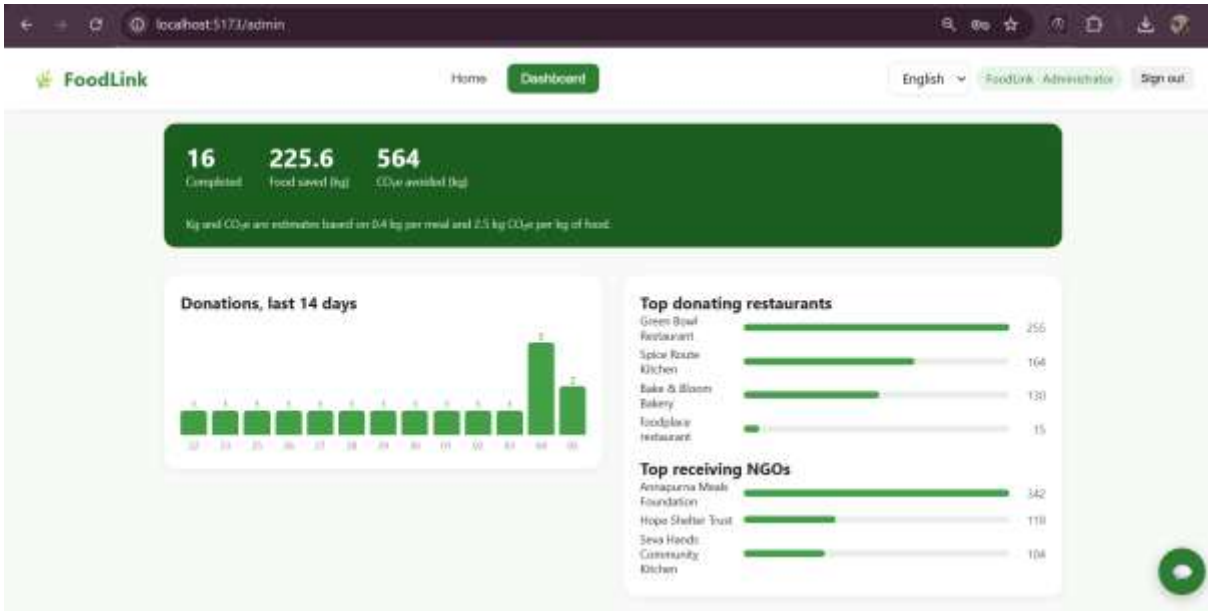


Fig 8.12 – Admin Dashboard – Platform statistics, user management, and audit log.

CHATBOT

A floating chat button, available on every page once the user is signed in, opens the FoodLink assistant. The assistant answers from live database data in the user's chosen language, and shows suggestion chips for the common questions: "What is expiring soon?", "Donation status", "Best NGO match", "My impact", and "Recently viewed". The answers depend on the role; for example, a restaurant is told which of its items expire soonest and which NGO is the best match for its next item, an NGO is shown its best open food, and an Admin is given platform totals. If an Anthropic API key is configured, questions that do not match any of these topics are answered by the Claude API using the user's dashboard summary as context; otherwise, the assistant politely lists what it can answer.

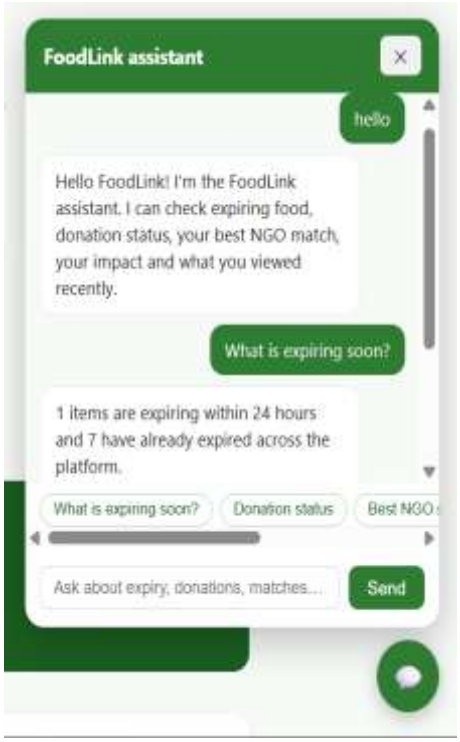


Fig 8.13 – FoodLink Assistant – Floating chatbot that answers from live data.

PROFILE, LANGUAGE, AND RECENTLY ACCESSED

The Profile page lets a user update the organization name, address, city, phone number, and location. A "Use my current location" button reads the coordinates from the browser, which SmartMatch needs to calculate distance and travel time. An NGO also sets its daily capacity in meals, its service radius in kilometers, the food types it accepts, and whether it has a vehicle for pickups. The user can switch the language between English, Hindi, and Tamil at any time; the choice is saved to the account and is also used by the chatbot and the scanner. Every food, NGO, or donation that a user opens is saved to the recently_accessed table (the latest 10 per user), and is shown as clickable shortcuts on both dashboards.

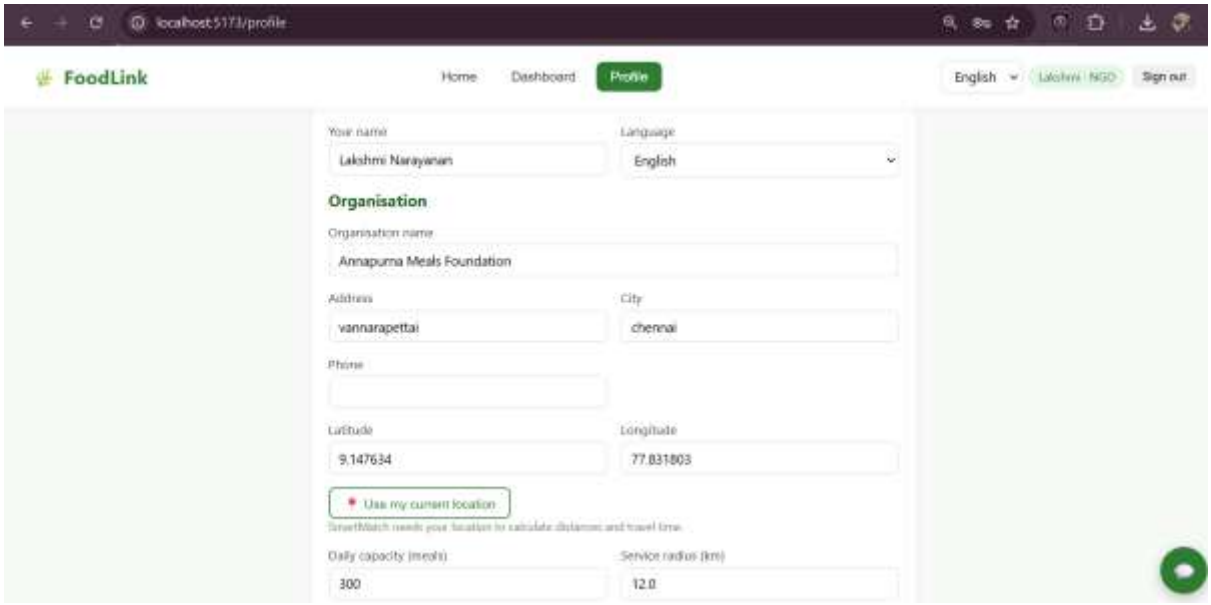


Fig 8.14 – Profile Page – Organization details, SmartMatch settings, and language.

8.3 SECURITY AND AUTHENTICATION

Passwords are hashed with bcrypt before they are stored. After login, the server issues a short-lived access token (15 minutes) and a refresh token (7 days). Only a hash of the refresh token is stored in the refresh_tokens table; the token is rotated every time it is used and is deleted on logout, so a logged-out session can never be refreshed. The authentication middleware verifies the access token on every protected request and then re-reads the user from MySQL, so a disabled account loses access immediately. The requireRole middleware restricts each endpoint to the roles that may use it. The Google OAuth flow uses a signed state value to protect against cross-site request forgery. Helmet adds secure HTTP headers, CORS allows only the frontend origin, login and registration are rate-limited, and all SQL uses parameterized queries.

8.4 TESTING

The project includes two automated test scripts that run against the live backend. The login check signs in every seeded account (one Admin, three Restaurants, and three NGOs), verifies the role inside each JWT, and checks that a wrong password, a forged token, and access to a forbidden route are rejected, and that token refresh and logout work. The workflow check

exercises the features that go beyond basic CRUD: validation of food listings, SmartMatch ranking and exclusion reasons, the full donation lifecycle, recently accessed tracking, the chatbot in all three languages, and the Admin controls.

CHAPTER 9

CONCLUSION

FoodLink provides a practical and effective solution for moving surplus food from restaurants to the people who need it, before it spoils. It reduces the need for manual coordination by showing restaurants which NGOs can actually collect their food in time, and by showing NGOs only the food they can realistically reach. By offering separate access for Admin, Restaurant, and NGO, the system ensures clear role-based access and control. Restaurants can list food, scan it for freshness, and track every donation; NGOs can claim and collect food within their capacity; and the Admin can monitor the whole platform and manage users.

The SmartMatch engine, the server-enforced donation lifecycle, the automatic expiry job, the AI Food Scanner, the live-data chatbot, and the three-language interface go beyond basic record keeping and make FoodLink a complete working system. The system helps maintain accurate records, improves transparency and security, saves time, and shows the impact of every donation in meals rescued. Overall, it is a valuable tool for restaurants and charitable organizations to reduce food waste in an organized and professional manner.